



Space Invaders: 2. Sprint-aren Patroien Txostena

Asier, Ibai, Lander, Oier

2026-ko apirilaren 13a

1 Sarrera

Txosten honen helburua Space Invaders jokoaren garapenaren bigarren sprintean inplementatu diren diseinu-patroien arkitektura eta justifikazioa aurkeztea da. Oinarrizko inplementaziotik haratago, fase honetan **Eredue-Bista-Kontroladore** (MVC) arkitektura finkatuta genduela, egitura eskalagarri bat sortzea izan da gure jomuga nagusia. Horretarako, software-ingeniaritzan estandarrek diren hainbat diseinu-patroi aplikatu ditugu. Patroi hauen bidez, domeinuaren logika eta interfaze grafikoa erabat banatzea lortu dugu, kodea askoz modularragoa, berrerabilgarriagoa eta mantentzeko errazagoa bihurtuz. Dokumentu honetan, jokoari dinamismoa eta sendotasuna emateko erabili den patroi bakoitzaren kokapena eta betetzen duen funtzio zehatza deskribatzen da.

2 Erabilitako Patroiak

2.1 Singleton Patroia

Proiektuaren oinarrizko kudeatzaile batzuentzat instantzia bakar bat egotea bermatu dugu.

- **Non agertzen da:** `JokoKudeatzailea`, `EtsaiFactory`, `JokalariOntziFactory` eta `MatrizeEredua` klaseetan inplementatu da. Klase hauetako bakoitzak bere instantzia propioa eskuratzeko `getXXX()` edo antzeko metodo estatiko bat dauka, eta euren eraikitzaileak pribatuak dira kanpotik instantzia berriak sortzea galarazteko.
- **Zertarako:** Jokoaren exekuzioan zehar osagai kritiko hauen bikoizketak saihestea. `MatrizeEredua`-ren kasuan, jokoaren logika eta datu-egitura nagusia denez, instantzia anitz egoteak sinkronizazio arazo larriak ekarriko lituzke. Era berean, `Factory`-ek ez dute zertan behin baino gehiagotan sortu memoria alferrik ez kontsumitzeko, eta `JokoKudeatzailea`-k jokoaren fluxu orokorra puntu zentralizatu bakar batetik kontrolatzen duela bermatzen dugu.

2.2 Factory patroia

Objektu asko sortu behar direnean, hobe da horretarako leku bakar bat izatea.

- **Non agertzen da:** `JokalariOntziFactory` eta `EtsaiFactory` klaseetan zehazki. Klase hauek sarrera-parametro batzuk jasotzen dituzte eta instantzia osatuak bueltatzen dituzte.
- **Zertarako:** Objektuen sorkuntza-logika konplexua ezkutatzeko eta jokoaren logika nagusia zehatz-mehatz zein klase konkretu sortzen ari den ez jakiteko. Espazio-ontzien kasuan, jokalariai aukeratutako kolorearen arabera parametro eta portaera ezberdinak dituen ontzia sortzen da. `Factory` patroia esker, etsai mota berri bat gehitu nahi badugu, nahikoa dugu `EtsaiFactory`-n baldintza bat gehitzea, jokoaren begizta nagusia edo matrizearen logika aldatu gabe.

2.3 Observer Patroia

`Eredua` eta `Bista` elkarrengandik independente mantentzea du helburu, baina uneoro sinkronizatuta egotea ziurtatuz. Horrela, jokoaren logika eta interfaze grafikoa ez daude bateratuta.

- **Non agertzen da:** MVC arkitekturaren Eredua eta Bista lotzeko erabiltzen da funtsean. Gure kodean, **Gelaxka** edo **MatrizeEredua** klaseak *Observable* (Eredua) gisa jokatzen dute, eta interfaze grafikoko elementuek, esaterako **GelaxkaBista** edo **LeihoNagusia** klaseek, *Observer* (Bista) interfazea inplementatzen dute.
- **Zertarako:** Ereduari eta interfaze grafikoaren arteko lotura ahalik eta gehien murrizteko. Eredua bere barne-logika exekutatzen du, eta egoera berri bat sortzen denean, Behatzaileei notifikatzen die. Bista, bestalde, Ereduari "entzuten"egongo da, eta mezu hori jasotzean bere burua eguneratuko du. Sistema honi esker, etorkizunean interfaze grafikoa erabat aldatzea erabakitzen badugu. Ereduko kodea ez genuke lerro bakar batean ere ukitu beharko.

2.4 Composite patroia

Patroi honen bidez, jokoaren elementu guztiak, jokalaria, etsaiak eta tiroak, modu berean tratatzea lortzen dugu, objektu bakarra izan ala multzo baten parte izan.

- **Non agertzen da:** Elementu bisual eta interaktiboen egituran sakonki integratuta dago: Jokalariaren Ontzia osatzen duten gelaxka anitzetan, **TiroKonposatua**-n, hainbat gelaxka hartzen dituzten tiro berezietan, eta Etsaien egituran, **EtsaiHostoa** eta **EtsaiNodoa**.
- **Zertarako:** Egitura hierarkikoak eraikitzeke eta bezeroari objektu konposatuak zein bakunak berdin tratatzeko ahalmena emateko. Etsai baten adibidea hartuta: etsaia pixel/gelaxka bakar batez osatuta egon daiteke, edo forma konplexua duen multzo bat izan daiteke. Patroi honen bidez, mugitzeko edo marrazteko agindua ematen diogunean, osagai bakar batek bere burua mugituko du, eta osagai anitzeko nodo batek agindua bere azpi-osagai guztiei delegatuko die. Horrela, jokoak ez du egiaztatu behar etsai batek zenbat atal dituen, logika nabarmen sinplifikatuz.

2.5 Strategy patroia

Objektu baten portaera, adibidez, nola mugitzen den edo nola egiten duen tiro, aldatu nahi badugu, portaera hori aldatzen diogu.

- **Non agertzen da:** Ontziaren formak kudeatzen dituen **OntziPortaera** interfazean (adibidez, **FormaBerdea**, **FormaGorria**) eta ontziaren eraso-motak definitzen dituen **TiroPortaera** interfazean (**TiroBakuna**, **TiroGezia**, **TiroErronboa**). Ontzi bakoitzak estrategia horien instantzia bat gordetzen du bere barruan.
- **Zertarako:** Klaseen portaera exekuzio-garaian modu dinamikoan aldatzeko aukera emateko, herentziaren zorrotasuna eta klase-leherketa saihestuz. Demagun ontzi batek botere berezi bat hartzen duela jokoan zehar; Strategy patroiarik esker, bere **TiroPortaera** aldagaiari estrategia berri bat esleitzen diezaiokegu erraz, ontzi klase berri bat sortu beharrik gabe. Honek jokoari malgutasun ikaragarria ematen dio arau eta portaera berriak era independentean kodetzeko.

2.6 State patroia

Gelaxka bakoitzak une bakoitzean duen egoeraren arabera jokatzen duela ziurtatzeko erabiltzen da, *EdukiaEgoera* egiturari esker.

- **Non agertzen da:** Taulako lauki bakoitza ordezkatzeko duen `Gelaxka` klasean dago integratuta. `Gelaxka` bakoitzak `EdukiaEgoera` motako atributu bat dauka, bere inplementazio ezberdinekin: `HutsaEgoera`, `EtsaiEgoera`, `HormaEgoera`, `EspazioOntziaEgoera` eta `TiroaEgoera`.
- **Zertarako:** `Gelaxka` bakoitzak une zehatz batean duen edukiaren arabera modu ezberdinean erantzuteko. `if-else` kate luze eta korapilatsuak idatzi beharrean, `Gelaxka` egungo egoerak berak kudeatzen du portaera, edo besterik gabe egoera berri batera trantsizioa egiten da, adibidez, etsaia hiltzean `EtsaiEgoera` izatetik `HutsaEgoera` izatera pasatzea. Kodea garbiagoa, irakurgarriagoa eta hedatzeko errazagoa bihurtzen du.

3 Ondorioa

Diseinu-patroien inplementazioak `Space Invaders` proiektuaren kalitatean jauzi kualitatibo garrantzitsu bat ematea ahalbidetu du. Alde batetik, `Singleton` eta `Observer` patroiek datuen koherentzia eta `MVC` arkitekturaren independentzia bermatu dute, domeinu-logika eta bista guztiz bananduz. Beste aldetik, `Factory`, `Composite`, `Strategy` eta `State` patroiek malgutasun ikaragarria eman diote sistemari. Egitura honi esker, entitate berriak, hala nola etsai mota konplexuagoak, tiro anitzak edo portaera berriak, txertatzea prozesu sinplea eta segurua da orain, ez baitu oinarritzko joko-begizta aldatzerik eskatzen. Laburbilduz, bigarren sprint honetan egindako berregituraketak kode garbi eta sendoa utzi digu, jokoaren etorkizuneko hedapenentarako oinarri tekniko ezin hobea finkatuz.